VEB DASTURLAR XAVFSIZLIGI

A11. Botlardan himoyalanish, monitoring va xavfsiz ekspluatatsiya

1-VARIANT — Login urinishlarini cheklash va xavfsizlik hodisalarini aniqlash

Mashg‘ulot maqsadi. Avtomatlashtirilgan login urinishlarini lokal xizmatda modellashtirish, IP va akkaunt bo‘yicha Rate Limitingni yoqish, xavfsizlik jurnalini tahlil qilish hamda cheklovdan keyin qonuniy foydalanuvchi uchun xizmat tiklanishini tekshirish.

Kutiladigan natija. Talaba 401 va 429 javoblarini farqlaydi, Retry-After sarlavhasini izohlaydi, mijoz bergan soxta IP sarlavhasiga tayanmaydi. JSON jurnalidan shubhali urinishlarni topadi, request_id orqali javobni hodisaga bog‘laydi, parol jurnalga yozilmaganini tekshiradi va dalillarni saqlab, qisqa javob choralarini tuzadi.

Ko‘rsatkich	Mashg‘ulot sharti
Fan / semestr	Veb dasturlar xavfsizligi / faqat 5-semestr
Mashg‘ulot	A11; 1-variant; individual ish
Auditoriya	30 ta kompyuter; lokal tarmoq va internet
Kompyuter	Windows 11; 8 yadroli CPU; RAM 32 GB; SSD 1 TB
Virtual muhit	VMware Workstation yoki VirtualBox; Ubuntu 24.04 LTS
VM resursi	2 vCPU, RAM 4 GB, disk 25 GB; NAT adapter
Vaqt / baho	80 daqiqa, tayyor muhitda / 10 ball

Vositalar va oldindan tayyorgarlik

1. Python 3, curl va matn muharriri; tashqi Python paketi, pullik xizmat yoki CAPTCHA akkaunti talab qilinmaydi.
2. Darsdan oldin VM, paketlar va ilovalardagi app.py hamda monitor.py tayyorlanadi. policy.json boshlang‘ich holati 1.1-bo‘limda berilgan. OT o‘rnatish va kodlarni Worddan ko‘chirish alohida vaqt oladi.

Ish chegarasi. Barcha so‘rovlar talabaning o‘z VMidagi 127.0.0.1:8011 manziliga yuboriladi. Faqat sun’iy akkaunt va parollar ishlatiladi. Mashg‘ulot kichik nazorat ketma-ketligidan iborat; ommaviy trafik yoki internetdagi akkauntlarni tekshirish talab qilinmaydi.

Arxitektura va himoya mantiqi

Bitta Ubuntu VMda curl mijoz va Python login xizmati ishlaydi. Xizmat faqat loopbackda tinglaydi. Ilova events.jsonl fayliga hodisalarni yozadi; monitor.py saqlangan jurnalni o‘qib, 60 soniyalik oynalarda shubhali ketma-ketliklarni aniqlaydi. Monitor tarmoqni skanerlamaydi va akkauntlarni bloklamaydi.

Qism	Vazifa	Dalil
POST /login	Kirish ma’lumotini tekshirish	200, 401 yoki 429
GET /health	Login cheklovidan mustaqil ishlash holati	200 / health_ok
events.jsonl	Har bir dastur hodisasi uchun bitta JSON satr	Vaqt, IP, user, event, status, request_id
monitor.py	Jurnaldan shubhali urinishlarni aniqlash	FAIL_BURST va LIMIT_BURST

Bir-biriga yaqin tushunchalar

Brute force bir akkauntga ko‘p parolni sinaydi; password spraying bir parolni ko‘p akkauntda sinaydi; credential stuffing esa boshqa joydan olingan login–parol juftlarini qayta tekshiradi. Mazkur variant takroriy noto‘g‘ri kirish va turli sun’iy nomlarga urinishni ko‘rsatadi; haqiqiy sizib chiqqan ma’lumotlar ishlatilmaydi. [1]

Ushbu variantning aniq cheklovlari

3. 30 soniyalik sirg‘aluvchi oynada bir akkauntga ko‘pi bilan 3 ta, bir IPga ko‘pi bilan 8 ta tekshiruvga ruxsat beriladi. Qabul qilingan muvaffaqiyatli va noto‘g‘ri urinishlar birday sanaladi.
4. Limitga tushgan so‘rov parolni tekshirishgacha rad etiladi. Bloklangan urinish oynani uzaytirmaydi; Retry-After ikkala hisoblagich bo‘shashishi uchun zarur vaqtni bildiradi.
5. Akkaunt nomi trim va lower orqali bir xillashtiriladi. IP TCP ulanishidan olinadi; foydalanuvchi yuborgan X-Forwarded-For bu modelda ishlatilmaydi.

Rate Limiting so‘rov tezligini boshqaradi, foydalanuvchining inson yoki botligini isbotlamaydi. NAT ortidagi qonuniy foydalanuvchilar bitta IPni ulashishi, taqsimlangan botlar esa turli IPlardan kelishi mumkin. MFA va zaruratga ko‘ra CAPTCHA qo‘shimcha himoya qatlamidir. [1]

Model chegarasi. Akkaunt alice, sun’iy parol Lab11-Only!; 200 faqat tekshiruv muvaffaqiyatini bildiradi, sessiya yaratmaydi. Koddagi ochiq demo parol real tizim uchun namuna emas: haqiqiy parollar mos parol xeshlash algoritmi bilan saqlanadi. [6]

1.1. Muhit va boshlang‘ich sozlamalar

1-qadam. Tayyor VM bo‘lmasa, Ubuntu 24.04 LTS ISO bilan 2 vCPU, 4 GB RAM, 25 GB disk va NAT adapterli VM yarating. OTni o‘rnatib, oddiy foydalanuvchi bilan kiring. Buyruqlar Ubuntu Bash uchun.

```
sudo apt update
sudo apt install -y python3 curl nano
umask 077
mkdir -p ~/a11-lab
cd ~/a11-lab
```

2-qadam. 1-ilovadagi A, B va C qismlarni ketma-ket bitta app.py fayliga, 2-ilovani monitor.py fayliga saqlang. UTF-8 va Python surilishlarini saqlang. Dastlab policy.json quyidagicha bo‘lsin:

```
cat > policy.json <<'EOF'
{
  "rate_limit": false,
  "window_seconds": 30,
  "ip_limit": 8,
  "account_limit": 3
}
EOF
cp policy.json policy.before.json
python3 -m json.tool policy.json
python3 -m py_compile app.py monitor.py
python3 --version
curl --version
touch events.jsonl
chmod 600 events.jsonl
python3 -u app.py
```

1-terminal ochiq qoladi. A11 127.0.0.1:8011 va faol policy qiymatlari chiqadi. Dastur sudo bilan ishga tushirilmaydi. 2-terminalda:

```
cd ~/a11-lab
U=http://127.0.0.1:8011
ss -ltn | grep ":8011"
curl --noproxy "*" -i "$U/health"
```

Kutiladigan natija: tinglovchi 127.0.0.1:8011; status 200; result=health_ok; X-Request-ID va Cache-Control: no-store. Javobdagi request_id har safar yangi bo‘ladi.

Dalil D1. VM parametrlari, versiyalar, faol policy, port va health javobi. Avvalgi tajribadan events.jsonl qolgan bo‘lsa, uni dastur ishga tushishidan oldin alohida nomga ko‘chiring; yangi mashg‘ulotni bo‘sh jurnal bilan boshlang.

1.2. Cheklovsiz holatni kuzatish

3-topshiriq. 2-terminalda atigi to‘rtta noto‘g‘ri urinish yuboring. So‘rov JSON formatida, barcha parollar sun’iy:

```
for attempt in 1 2 3 4; do
  curl --noproxy "*" -s -i "$U/login" \
    -H "Content-Type: application/json" \
    -d '{"username":"alice","password":"Wrong11!"}'
done
```

To‘rtala so‘rov 401 / login_failed. To‘rtinchi urinish ham parol tekshirishga yetadi: rate_limit=false. So‘ng mavjud bo‘lmagan nom va to‘g‘ri demo parol bilan nazorat qiling:

```
curl --noproxy "*" -i "$U/login" \
  -H "Content-Type: application/json" \
  -d '{"username":"nobody","password":"Wrong11!"}'
curl --noproxy "*" -i "$U/login" \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"Lab11-Only!"}'
```

Sinov	Status	result
Alice, noto‘g‘ri parol, 1–4-urinish	401	login_failed
Noma’lum nobody nomi	401	login_failed
Alice, to‘g‘ri demo parol	200	login_ok

Noto‘g‘ri parol va noma’lum nom uchun umumiy xabar qaytadi. request_idlari farqli bo‘lishi tabiiy. Bu sinov javob matnidan akkaunt mavjudligini ochiq bildirmaslikni tekshiradi; vaqt kanali orqali aniqlash yo‘qligini isbotlamaydi.

Boshlang‘ich dalilni saqlash

1-terminalda Ctrl+C bosing. Server to‘xtaganidan keyin quyidagilarni bajaring; jurnalni yozilayotgan paytda ko‘chirmang:

```
mv events.jsonl before.jsonl
chmod 400 before.jsonl
python3 monitor.py before.jsonl | tee before-alerts.txt
```

Ketma-ketlik 60 soniya ichida bajarilgan bo‘lsa FAIL_BURST chiqadi: bitta IPdan kamida 3 ta login_failed. Hali rate_limited yo‘q. Aynan yuqoridagi ketma-ketlikda 5 ta login_failed, 1 ta login_ok va D1dagi 1 ta health_ok bor; qo‘shimcha so‘rovlar sonlarni o‘zgartiradi.

Dalil D2. To‘rtta 401, nazorat 200, before.jsonl va monitor chiqishi. Ogohlantirish shubhali tezlikni bildiradi; bitta IP yoki noto‘g‘ri parollar foydalanuvchining zararli niyatini o‘zi isbotlamaydi.

1.3. Akkaunt limiti va ishonchli IP manbasi

4-topshiriq. Server to‘xtagan holatda policy.jsondagi faqat rate_limit qiymatini true qiling. Qolgan qiymatlar 30, 8 va 3 bo‘lib qolsin. JSONda true kichik harf bilan yoziladi.

```
nano policy.json
python3 -m json.tool policy.json
diff -u policy.before.json policy.json
cp policy.json policy.good.json
python3 -u app.py
```

diff farq topganda chiqish kodi 1 bo‘lishi tabiiy. Faol policy ichida rate_limit=True ko‘rinsin. Yangi events.jsonl avtomatik yaratiladi. 2-terminalda to‘rtta urinishni tez ketma-ket bajaring:

```
for attempt in 1 2 3 4; do
  curl --noproxy "*" -s -i "$U/login" \
    -H "Content-Type: application/json" \
    -d '{"username":"alice","password":"Wrong11!"}'
done
```

Kutiladigan natija: 401, 401, 401, 429. To‘rtinchi javob result=rate_limited; Retry-After odatda 1–30 soniya. 429 juda ko‘p so‘rovni, 401 esa tekshiruvdan o‘tmagan kirishni bildiradi. Retry-After server qachondan keyin qayta urinishni taklif qilishini ko‘rsatadi. [4]

Soxta X-Forwarded-Forni nazorat qilish

Oyna tugamasidan quyidagini yuboring. Parol to‘g‘ri bo‘lsa ham limit tekshiruvi undan oldin ishlaydi:

```
curl --noproxy "*" -i "$U/login" \
  -H "Content-Type: application/json" \
  -H "X-Forwarded-For: 198.51.100.10" \
  -d '{"username":"alice","password":"Lab11-Only!"}'
curl --noproxy "*" -i "$U/health"
```

Login 429, health 200. Jurnaldagi IP 127.0.0.1 bo‘lib qoladi. 198.51.100.10 — faqat test sarlavhasi; u manzilga ulanish yuborilmaydi. Haqiqiy reverse proxy ortida mijoz IPsi faqat ishonchli proksi zanjiridan olinadi; har qanday X-Forwarded-Forni qabul qilish limitni chetlab o‘tishga sabab bo‘lishi mumkin.

Dalil D3. 401×3 → 429, Retry-After, soxta IP bilan 429 va health 200. Sinov davomida 30 soniya o‘tib ketsa, muddat tugashi natijani o‘zgartiradi; yangi oynada butun ketma-ketlikni qaytaring.

1.4. Qonuniy tiklanish va IP bo‘yicha cheklov

5-topshiriq. Akkaunt cheklovi doimiy emasligini tekshiring. Oxirgi urinishdan keyin 31 soniya kuting; kutish davomida login so‘rovi yubormang:

```
sleep 31
curl --noproxy "*" -i "$U/login" \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"Lab11-Only!"}'
```

Natija 200 / login_ok. 31 soniya ushbu 30 soniyalik o‘quv oynasi uchun tanlangan; amaliy mijoz Retry-Afterga amal qiladi. Bloklangan urinishlarning oynani uzaytirmasligi tiklanish hisobini soddalashtiradi.

6-topshiriq. Yana 31 soniya kutib, eski IP hisoblagichi bo‘shashganidan keyin turli sun’iy nomlar bilan 9 ta urinish yuboring:

```
sleep 31
for n in 1 2 3 4 5 6 7 8 9; do
  curl --noproxy "*" -s -i "$U/login" \
    -H "Content-Type: application/json" \
    -d '{"username":"user'"$n"'","password":"Wrong11!"}'
done
```

Birinchi 8 tasi 401, 9-si 429. Har akkauntga faqat bir urinish bo‘lgani uchun bu safar IP limiti ishga tushadi. Nomlar bazada mavjud emas; bu mavjud bo‘lmagan nomlar ham hisoblanishini ko‘rsatadi. 9 ta so‘rovdan keyin to‘xtang.

CAPTCHA va MFA bo‘yicha qisqa qaror

Quyidagi holatlar uchun javob chorasi yozing. Bu tahliliy topshiriq; CAPTCHA integratsiyasi kodda yo‘q. CAPTCHA avtomatlashtirishni qiyinlashtirishi mumkin, lekin uni mutlaq kafolat deb bo‘lmaydi. Xavf oshganda qo‘shimcha tekshiruv va MFA birgalikda rejalashtiriladi. [1]

Holat	Talabaning qarori
Bir marta parolni adashtirgan odatiy foydalanuvchi	_____
Takroriy urinish va ko‘p akkauntga bir xil manbadan murojaat	_____
CAPTCHA tokeni yo‘q, muddati o‘tgan yoki qayta yuborilgan	_____

Token ishlatilsa, tasdiq server tomonida bajarilishi, token amal qilish muddati va qayta ishlatilishi tekshirilishi kerak. Faqat brauzerda “tasdiqlandi” belgisini ko‘rsatish server tekshiruvini almashtirmaydi.

Dalil D4. Muddatdan keyingi 200, turli nomlar bo‘yicha 401×8 → 429 va CAPTCHA/MFA qarori. IP/akkaunt limitlari noto‘g‘ri sozlansa qonuniy foydalanuvchiga xizmatni rad etish vositasiga ham aylanishi mumkin.

1.5. Jurnal va monitoring natijalarini tekshirish

7-topshiriq. 1-terminalda Ctrl+C bilan serverni to‘xtating. Jurnal yozilishi tugagach, nusxa oling va monitoringni bajaring:

```
cp events.jsonl after.jsonl
chmod 400 after.jsonl
python3 monitor.py after.jsonl | tee after-alerts.txt
tail -n 6 after.jsonl
stat -c "%a %n" events.jsonl before.jsonl after.jsonl
```

events.jsonl huquqi 600, dalil nusxalari 400. Jurnalda ts — Unix vaqti soniyalarda, request_id — javob bilan bog‘lovchi identifikator, ip — TCP manbasi, user — tekshirilgan sun’iy nom, event va status mavjud.

Qoida	Shart	Talqin
FAIL_BURST	60 soniyada bir IPdan ≥3 login_failed	Ko‘p muvaffaqiyatsiz tekshiruv
LIMIT_BURST	60 soniyada bir IPdan ≥2 rate_limited	Faol limitga takror urilish
invalid_lines	O‘qilmagan yoki sxemasi buzilgan satrlar	0 bo‘lishi kutiladi; aks holda tahlil to‘liq emas

D3 tez bajarilgan bo‘lsa ikkala ogohlantirish ham chiqadi. Monitor butun faylni o‘qiydi, vaqt bo‘yicha tartiblaydi va har IP–qoida juftligini bitta ishga tushirishda bir marta chiqaradi. Bu real vaqt SIEMi emas; keyingi mustaqil hodisalarni alohida sanash va avtomatik xabar yuborish bajarilmagan.

Maxfiy qiymat yozilmaganini tekshirish

```
grep -nE 'Wrong11!|Lab11-Only!|password|Authorization' after.jsonl
```

Chiqish bo‘lmasin; grepning 1 kodi bu yerda moslik topilmaganini bildiradi. Faqat shu test qiymatlari bo‘yicha tekshiruv bajarildi, barcha mumkin bo‘lgan maxfiy ma’lumotlar yo‘qligi isbotlanmadi. Parol, sessiya tokeni va to‘liq so‘rov tanasi jurnalga yozilmaydi. [2]

D3dagi bitta 429 javobining X-Request-ID qiymatini ko‘chiring va quyidagi buyruqda almashtiring:

```
grep -F "BU_YERGA_REQUEST_ID" after.jsonl
```

Aynan shu identifikatorli yozuvda status=429, event=rate_limited va ip=127.0.0.1 bo‘lsin. Ogohlantirishdagi request_id chegara bajarilgan hodisaga bog‘langan.

Dalil D5. Ikki ogohlantirish, invalid_lines=0, maxfiy qiymatlar qidiruvi, fayl huquqlari va request_id orqali mos yozuv. Log faylining o‘zi ishonchli manba bo‘lishi uchun kirish huquqi, saqlash muddati va yaxlitligi ham boshqariladi.

1.6. Xavfsiz ekspluatatsiya va hodisaga javob

8-topshiriq. Dastur to‘xtagan holda konfiguratsiya va dalil nusxalarini saqlang. Oldingi nusxalarni yo‘qotmaslik uchun har tajribada alohida katalogdan foydalaning:

```
CASE_DIR="evidence-$(date -u +%Y%m%dT%H%M%SZ)"
mkdir -m 700 "$CASE_DIR"
cp before.jsonl after.jsonl before-alerts.txt after-alerts.txt \
  policy.before.json policy.good.json "$CASE_DIR/"
sha256sum "$CASE_DIR"/* > "$CASE_DIR/SHA256SUMS"
sha256sum -c "$CASE_DIR/SHA256SUMS"
chmod 400 "$CASE_DIR"/*
cp policy.good.json policy.restore-check.json
cmp policy.good.json policy.restore-check.json
```

Hash tekshiruvida har bir fayl OK, cmpda chiqish yo‘q va kod 0. Bu nusxaning ayni paytdagi bayt mosligini tekshiradi. Manifestni ham almashtira oladigan foydalanuvchidan himoya yoki mustaqil vaqt tamg‘asi isbotlanmagan. Dalil saqlash buyruqlari bir katalog uchun bir marta bajarilsin.

Hodisa kartasi — 5–7 gap bilan to‘ldiring

6. Aniqlash: qaysi qoida, vaqt oralig‘i, IP va request_id shubha uyg‘otdi? Muvaffaqiyatli login bo‘lganmi?
7. Tekshirish: foydalanuvchining xatosi, umumiy NAT yoki rejalashtirilgan test bo‘lishi mumkinmi? Qaysi qo‘shimcha dalil kerak?
8. Cheklash: limitni nazorat ostida saqlash; zarur bo‘lsa xavfga mos CAPTCHA/MFA. Faqat alert asosida akkaunt buzilgan deb xulosa qilmaslik.
9. Tiklash: noto‘g‘ri konfiguratsiya bo‘lsa policy.good.jsondan qaytish, restart va health hamda loginni qayta tekshirish. Dalillarni saqlash.

Konfiguratsiyani qaytarish tartibi

```
cp policy.good.json policy.json
python3 -m json.tool policy.json
python3 -u app.py
```

Boshqa terminalda /health 200 va bitta to‘g‘ri login 200ni tekshiring. Keyin serverni Ctrl+C bilan to‘xtating. Restart ushbu modelning xotiradagi limit hisoblagichlarini ham tozalaydi; ishlab chiqarishda bu cheklovni chetlab o‘tish imkoniga aylanmasligi kerak.

Dalil D6. SHA256SUMS tekshiruvi, konfiguratsiya nusxasi, hodisa kartasi va qayta ishga tushgandan keyingi health/login. Real tizimda umumiy limit ombori, atomar hisoblash, jurnal rotatsiyasi, disk nazorati va yangilash jarayoni alohida talab qilinadi. [2, 3]

Ushbu HTTPServer o‘quv uchun. Slowloris, katta trafik, ko‘p jarayon, cheksiz akkaunt kalitlari va log diski to‘lishidan kompleks himoya yo‘q; uni internetga joylashtirmang. Python hujjati ham http.serverni ishlab chiqarish uchun tavsiya qilmaydi. [5]

1.7. Vaqt rejasi, hisobot va baholash

Bosqich	Vaqt	Dalil
Muhit va xizmatni boshlash	8 daqiqa	D1
Cheklovsiz nazorat va jurnal	10 daqiqa	D2
Akkaunt limiti va soxta IP	14 daqiqa	D3
Tiklanish, IP limiti, CAPTCHA qarori	12 daqiqa	D4
Jurnal va monitoring	15 daqiqa	D5
Dalillar va xavfsiz qaytarish	11 daqiqa	D6
Hisobot va himoya	10 daqiqa	Yakuniy xulosa
Jami	80 daqiqa	Tayyorgarlik alohida

Hisobot tarkibi

Titulda fan, 5-semestr, A11, 1-variant, F.I.Sh., guruh va sana ko‘rsatiladi. VM/dastur versiyalari, arxitektura, D1–D6 dalillari, boshlang‘ich va yakuniy policy, monitoring chiqishi hamda hodisa kartasi ilova qilinadi. Har bir dalilda buyruq va natija birga ko‘rinsin.

Tekshiruv	Kutiladigan holat	Amaldagi natija
Limit o‘chiq: 4 ta noto‘g‘ri login	401 × 4	_____
Akkaunt limiti yoqilgan	401 × 3 → 429	_____
Soxta IP bilan limitdan o‘tish	429; haqiqiy IP saqlanadi	_____
Muddat tugagach qonuniy login	200	_____
Turli 9 nom; bitta IP	401 × 8 → 429	_____
Monitoring	FAIL_BURST, LIMIT_BURST	_____

Baholash mezonlari

Mezon	Ball
Muhit, health va boshlang‘ich nazorat	1
Akkaunt limiti, Retry-After va soxta IP tekshiruvi	2
IP limiti va qonuniy tiklanish	2
Monitoring, request_id va maxfiy qiymat tekshiruvi	2
Dalil nusxalari, hash va konfiguratsiyani qaytarish	2
Hisobot, CAPTCHA/MFA qarori va xulosa	1
Jami	10

Qabul sharti. 429ning o‘zi yetarli emas: limit yo‘qligidagi holat, muddatdan keyingi qonuniy 200, healthning ishlashi va jurnal bilan moslik ko‘rsatilishi kerak. Parollarni skrinshotlarda ko‘rsatish faqat mazkur sun’iy qiymatlar uchun; real maxfiy qiymatlar ishlatilmaydi.

Muammolar, nazorat savollari va manbalar

Belgi	Tekshirish / yechim
Connection refused yoki port band	app.py ishlayaptimi? Oldingi jarayon terminalida Ctrl+C bosing.
To‘rtinchi urinish ham 401	Faol rate_limit true ekanini tekshiring; tahrirdan so‘ng restart kerak.
Kutilganidan oldin 429	Oynada oldingi urinishlar qolgan; 31 soniya tinch kutib qaytaring.
Login 400 yoki 413	JSON qo‘shtirnoqlari, username turi/uzunligi va tana hajmini tekshiring.
Alert yo‘q	Hodisalar bir IPdan 60 soniya ichidami? Kerakli jurnal tanlanganmi?
invalid_lines > 0	To‘liq bo‘lmagan yoki buzilgan JSON satri bor; nusxani server to‘xtagach oling.
Permission denied	Faol events.jsonl 600, arxiv nusxa 400 bo‘lsin; server oddiy egasi bilan ishlasin.

Nazorat savollari

10. 401 va 429 o‘rtasidagi farq nima? Nega to‘g‘ri parol ham vaqtincha 429 oladi?
11. IP va akkaunt bo‘yicha limitlarni birga qo‘llash nimani yaxshilaydi?
12. CAPTCHA nega yakka o‘zi yetarli emas va nima uchun server tasdig‘i kerak?
13. request_id, vaqt va hodisa turi insident tahlilida qanday bog‘lanadi?
14. Restart hisoblagichni tozalashi va umumiy NAT qanday cheklovlarni keltirib chiqaradi?

Rasmiy manbalar

[1] OWASP — Credential Stuffing Prevention

[2] OWASP — Logging Cheat Sheet

[3] OWASP — Denial of Service Cheat Sheet

[4] RFC 6585 — 429 Too Many Requests

[5] Python — http.server

[6] OWASP — Password Storage Cheat Sheet

Manbalar tekshirilgan sana: 07.09.2026. Mavzu taqdim etilgan fan dasturining A11 bandiga mos.

O‘qituvchi uchun. Fayllarni oldindan tarqating; eski jurnallarni ajrating va rate_limit=false bilan boshlang. Tayyorlashda 27 ta lokal HTTP tekshiruvi, limit oynasining tugashi, soxta IP, jurnal tarkibi va monitoring qoidalari tekshirildi. CAPTCHA, real MFA va ishlab chiqarish muhiti sinovi bajarilmagan.

1-ilova. app.py — A qism

A, B va C qismlarni ketma-ket bitta ~/a11-lab/app.py fayliga birlashtiring. Sarlavhalarni ko‘chirmang; Python surilishlarini saqlang.

```
import json
import math
import os
import re
import time
import uuid
import hmac
from collections import defaultdict, deque
from pathlib import Path
from http.server import BaseHTTPRequestHandler, HTTPServer

ROOT = Path(__file__).resolve().parent
CFG = json.loads((ROOT / 'policy.json').read_text())
BUCKETS = defaultdict(deque)
CLOCK = time.monotonic
os.umask(0o077)
LOG = ROOT / 'events.jsonl'
# Synthetic fixture only; no session or real password database.
DEMO_PASSWORD = 'Lab11-Only!'


def retry_after(ip, user):
    now, window = CLOCK(), CFG['window_seconds']
    pairs = [(('ip', ip), CFG['ip_limit']),
             (('account', user), CFG['account_limit'])]
    waits = []
    for key, limit in pairs:
        queue = BUCKETS[key]
        while queue and queue[0] <= now - window:
            queue.popleft()
        if len(queue) >= limit:
            waits.append(queue[0] + window - now)
    if waits:
        return max(1, math.ceil(max(waits)))
    for key, limit in pairs:
        BUCKETS[key].append(now)
    return 0
```

Hisoblagichlar sirg‘aluvchi oynada faqat ruxsat berilgan tekshiruvlarni saqlaydi. Bloklangan so‘rov ularga qo‘shilmaydi. Ip va akkaunt bo‘yicha ikkala talab bajarilganda tekshiruvga yo‘l ochiladi.

1-ilova. app.py — B qism

A, B va C qismlarni ketma-ket bitta ~/a11-lab/app.py fayliga birlashtiring. Sarlavhalarni ko‘chirmang; Python surilishlarini saqlang.

```
class App(BaseHTTPRequestHandler):
    def log_message(self, *args):
        pass

    def send_result(self, status, event, user='-', retry=0):
        rid = uuid.uuid4().hex
        record = dict(ts=round(time.time(), 3), request_id=rid,
                      ip=self.client_address[0], user=user,
                      event=event, status=status)
        with LOG.open('a', encoding='utf-8') as out:
            out.write(json.dumps(record) + '\n')
        body = json.dumps({'result': event, 'request_id': rid}).encode()
        self.send_response(status)
        self.send_header('Content-Type', 'application/json')
        self.send_header('Content-Length', str(len(body)))
        self.send_header('Cache-Control', 'no-store')
        self.send_header('X-Request-ID', rid)
        if retry:
            self.send_header('Retry-After', str(retry))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        if self.path == '/health':
            self.send_result(200, 'health_ok')
        else:
            self.send_result(404, 'not_found')

    def do_POST(self):
        if self.path != '/login':
            return self.send_result(404, 'not_found')
        try:
            length = int(self.headers.get('Content-Length', '0'))
```

Jurnal uchun faqat oldindan tanlangan maydonlar yoziladi. So‘rov tanasi va sarlavhalari nusxalanmaydi. JSONning to‘g‘ri yozilishi keyingi monitor bosqichi uchun muhim.

1-ilova. app.py — C qism

A, B va C qismlarni ketma-ket bitta ~/a11-lab/app.py fayliga birlashtiring. Sarlavhalarni ko‘chirmang; Python surilishlarini saqlang.

```
            if not 0 < length <= 1024:
                return self.send_result(413, 'invalid_size')
            data = json.loads(self.rfile.read(length))
            user = data.get('username', '').strip().lower()
            password = data.get('password')
            if not re.fullmatch(r'[a-z0-9_]{1,24}', user):
                raise ValueError('invalid username')
            if not isinstance(password, str) or len(password) > 128:
                raise ValueError('invalid password')
        except (ValueError, AttributeError, TypeError):
            return self.send_result(400, 'invalid_input')
        wait = retry_after(self.client_address[0], user) if (
            CFG['rate_limit']) else 0
        if wait:
            return self.send_result(429, 'rate_limited', user, wait)
        valid = hmac.compare_digest(password.encode(), DEMO_PASSWORD.encode())
        valid = valid and user == 'alice'
        self.send_result(200 if valid else 401,
                         'login_ok' if valid else 'login_failed', user)


if __name__ == '__main__':
    server = HTTPServer(('127.0.0.1', 8011), App)
    print('A11 127.0.0.1:8011; policy=', CFG, flush=True)
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        pass
    finally:
        server.server_close()
```

Username normallashtiriladi; noto‘g‘ri format 400 bilan rad etiladi. Dastur parol tekshiruvining o‘quv modeli bo‘lib, sessiya yoki token bermaydi.

2-ilova. monitor.py

~/a11-lab/monitor.py fayliga saqlang. Ishga tushirish: python3 monitor.py after.jsonl. Natija JSON satrlari ko‘rinishida chiqariladi.

```
import json
import sys
from collections import Counter, defaultdict, deque
from pathlib import Path

rows, invalid = [], 0
for line in Path(sys.argv[1]).read_text().splitlines():
    try:
        row = json.loads(line)
        assert isinstance(row['ts'], (int, float))
        assert all(isinstance(row[k], str) for k in
                   ('ip', 'user', 'event', 'request_id'))
        rows.append(row)
    except (ValueError, KeyError, TypeError, AssertionError):
        invalid += 1
queues, seen = defaultdict(deque), set()
totals = Counter(row['event'] for row in rows)
for row in sorted(rows, key=lambda r: r['ts']):
    if row['event'] not in ('login_failed', 'rate_limited'):
        continue
    queue = queues[row['ip']]
    queue.append(row)
    while queue and queue[0]['ts'] < row['ts'] - 60:
        queue.popleft()
    counts = Counter(r['event'] for r in queue)
    for name, event, threshold in (
        ('FAIL_BURST', 'login_failed', 3),
        ('LIMIT_BURST', 'rate_limited', 2)
    ):
        key = (row['ip'], name)
        if counts[event] >= threshold and key not in seen:
            seen.add(key)
            print(json.dumps(dict(alert=name, ip=row['ip'],
                count=counts[event], window_seconds=60,
                request_id=row['request_id'], ts=row['ts'])))
print(json.dumps(dict(summary=dict(totals), invalid_lines=invalid)))
```

Talqin chegarasi

Bir IP–qoida juftligi bir ishga tushirishda bir marta chiqariladi. Bu dars hisobotidagi takror ogohlantirishlarni kamaytiradi; uzoq muddatli monitoringda hodisalarni yopish, yangi davrni boshlash va bildirishnomalar alohida loyihalanadi.